

Assignment_3.4

2303A51512

B_26

Assignment Number: 3.4 (Present assignment number)/24(Total number of assignments)		
Q.No.	Question	Expected Time to complete
1	Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques Task 1: Zero-shot Prompt – Fibonacci Series Generator Task Description #1 • Without giving an example, write a single comment prompt asking GitHub Copilot to generate a Python function to print the first N Fibonacci numbers. Expected Output #1 • A complete Python function generated by Copilot without any example provided. • Correct output for sample input N = 7 → 0 1 1 2 3 5 8 • Observation on how Copilot understood the instruction with zero context.	Week2

```
h.py optimize_prime_no_checking.py prime check in functions.py algorithm.py.py fibonacci.py Untitled-1
C:\Users\prade> OneDrive > Pradeep > fibonacci.py > ...
1 def print_fibonacci(n):
2     """
3     Print the first N Fibonacci numbers.
4
5     Args:
6         n (int): The number of Fibonacci numbers to print.
7     """
8     if n <= 0:
9         print("Please enter a positive integer.")
10        return
11
12    a, b = 0, 1
13    for i in range(n):
14        print(a, end=' ')
15        a, b = b, a + b
16    print()
17
18
19 if __name__ == "__main__":
20     n = int(input("Enter the number of Fibonacci numbers to print: ")) # Change this value to print a different number
21     print_fibonacci(n)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

```
.py
0 1 1 2 3 5 8 13 21 34

C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe c:\Users\prade\OneDrive\Pradeep\fibonacci
.py
0 1 1 2 3 5 8 13 21 34
```

1. Fibonacci Function (Earlier)

- **Purpose:** Print the first N Fibonacci numbers
- **Function:** [print_fibonacci\(n\)](#)
- **Logic:** Uses two variables (a, b) to track consecutive Fibonacci numbers, iterates N times
- **Example:** print_fibonacci(10) prints: 0 1 1 2 3 5 8 13 21 34

Task 2: One-shot Prompt – List Reversal Function

Task Description #2

- Write a comment prompt to reverse a list and provide one example below the comment to guide Copilot.

Expected Output #2

- Copilot-generated function to reverse a list using slicing or loop.
- Output: [3, 2, 1] for input [1, 2, 3]
- Observation on how adding a single example improved Copilot's accuracy.

```
factorial.py  prime num.py  optimize_prime_no_checking.py  prime check in functions.py  algorithm.py.py  reverse.py X
C:\Users\prade> prade > OneDrive > Pradeep > reverse.py > ...
1 def reverse_list_slicing(lst):
2     """
3     Reverse a list using slicing.
4
5     Args:
6         lst (list): The list to reverse.
7
8     Returns:
9         list: The reversed list.
10    """
11    return lst[::-1]
12
13
14 def reverse_list_loop(lst):
15     """
16     Reverse a list using a loop.
17
18     Args:
19         lst (list): The list to reverse.
20
21     Returns:
22         list: The reversed list.
23    """
24    reversed_list = []
25    for i in range(len(lst) - 1, -1, -1):
26        reversed_list.append(lst[i])
27    return reversed_list
28
29
30 if __name__ == "__main__":
31     # Test cases
32     test_list = [1, 2, 3]
33     print(f"Original list: {test_list}")
34     print(f"Reversed using slicing: {reverse_list_slicing(test_list)}")
35     print(f"Reversed using loop: {reverse_list_loop(test_list)}")
36
37
38 PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER ⓘ
C:\Users\prade> C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe c:\Users\prade\OneDrive\Pradeep\reverse.py
Original list: [1, 2, 3]
Reversed using slicing: [3, 2, 1]
Reversed using loop: [3, 2, 1]
```

List Reversal Functions (Earlier)

- Two implementations:
 - [reverse_list_slicing\(lst\)](#) - Uses Python slicing [::-1]
 - [reverse_list_loop\(lst\)](#) - Uses backward loop with range
- Example: Input [1, 2, 3] → Output [3, 2, 1]

Task 3: Few-shot Prompt – String Pattern Matching

Task Description #3

- Write a comment with 2–3 examples to help Copilot understand how to check if a string starts with a capital letter and ends with a period.

Expected Output #3

- A function is `_valid()` that checks the pattern.
- Output: True or False based on input.
- Students reflect on how multiple examples guide Copilot to generate more accurate code.

```

C:\Users\prade > OneDrive > Pradeep > reverse.py > is_valid
1  import re
2
3
4  def is_valid(pattern, text):
5      """
6      Check if text matches a given pattern.
7
8      Args:
9      pattern (str): The regex pattern to match against.
10     text (str): The text to validate.
11
12     Returns:
13     bool: True if text matches the pattern, False otherwise.
14     """
15     try:
16         return bool(re.match(pattern, text))
17     except re.error:
18         return False
19
20
21 if __name__ == "__main__":
22     # Example 1: Email pattern validation
23     print("--- Email Pattern Validation ---")
24     email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
25     emails = [
26         ("user@example.com", True),
27         ("john.doe@company.co.uk", True),
28         ("invalid.email@", False),
29         ("@domain.com", False),
30         ("no-at-sign.com", False),
31     ]
32
33     for email, expected in emails:
34         result = is_valid(email_pattern, email)
35         status = "✓" if result == expected else "X"
36         print(f"{status} {email:<30} -> {result} (expected {expected})")
37
38     # Example 2: Phone number pattern validation
39     print("\n--- Phone Number Pattern Validation ---")
40     phone_pattern = r'^\d{3}-\d{3}-\d{4}$'
41     phones = [
42         ("123-456-7890", True),
43         ("555-123-4567", True),
44         ("1234567890", False),
45         ("123-45-6789", False),
46     ]
47
48     for phone, expected in phones:
49         result = is_valid(phone_pattern, phone)
50         status = "✓" if result == expected else "X"
51         print(f"{status} {phone:<30} -> {result} (expected {expected})")
52
53     # Example 3: Alphanumeric pattern validation
54     print("\n--- Alphanumeric Pattern Validation ---")
55     alphanum_pattern = r'^[a-zA-Z0-9]+$'
56     texts = [
57         ("abc123", True),
58         ("Test2024", True),
59         ("hello-world", False),
60         ("with spaces", False),
61     ]
62
63     for text, expected in texts:
64         result = is_valid(alphanum_pattern, text)
65         status = "✓" if result == expected else "X"
66         print(f"{status} {text:<30} -> {result} (expected {expected})")
67

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER 3

```

--- Alphanumeric Pattern Validation ---
✓ abc123                -> True (expected True)
✓ Test2024              -> True (expected True)
✓ hello-world           -> False (expected False)
✓ with spaces           -> False (expected False)

```

Email Validation - Pattern Checking (Earlier)

- **Function:** `is_valid(pattern, text)`

- **Purpose:** Validates text against regex patterns
- **Demonstrated with 3 patterns:**
 - Email pattern validation
 - Phone number validation (XXX-XXX-XXXX format)
 - Alphanumeric validation
- **Returns:** True or False based on pattern match

Task 4: Zero-shot vs Few-shot – Email Validator

Task Description #4

- First, prompt Copilot to write an email validation function using zero-shot (just the task in comment).
- Then, rewrite the prompt using few-shot examples.

Expected Output #4

- Compare both outputs:

Zero-shot may result in basic or generic validation.

Few-shot gives detailed and specific logic (e.g., @ and domain checking).

- Submit both code versions and note how few-shot improves reliability.

```
C:\Users\prade > OneDrive > Pradeep > reverse.py > ...
1 # =====
2 # ZERO-SHOT APPROACH
3 # =====
4 # Write a function to validate email addresses
5
6 def validate_email_zero_shot(email):
7     """Email validation using zero-shot (basic approach)."""
8     if '@' in email and '.' in email:
9         return True
10    return False
11
12
13 # =====
14 # FEW-SHOT APPROACH
15 # =====
16 # Write a function to validate email addresses.
17 # Examples:
18 # - "user@example.com" -> True (valid: has username, @, domain, dot, extension)
19 # - "john.doe@company.co.uk" -> True (valid: supports subdomains and multiple dots)
20 # - "invalid@" -> False (invalid: missing domain)
21 # - "@domain.com" -> False (invalid: missing username)
22 # - "user@domain" -> False (invalid: missing extension after dot)
23 # - "user.name@domain.com" -> False (invalid: consecutive dots in username)
24 # - "user.com" -> False (invalid: empty domain part)
25
26 def validate_email_few_shot(email):
27     """Email validation using few-shot (detailed approach)."""
28     # Check if email is a string
29     if not isinstance(email, str):
30         return False
31
32     # Check for exactly one @ symbol
33     if email.count('@') != 1:
34         return False
35
```

```
C:\Users\prade > prade > OneDrive > Pradeep > reverse.py > ...
26 def validate_email_few_shot(email):
27
28     # Split into username and domain
29     username, domain = email.split('@')
30
31     # Check if username and domain are not empty
32     if not username or not domain:
33         return False
34
35     # Username cannot have consecutive dots
36     if '..' in username:
37         return False
38
39     # Check if domain contains at least one dot
40     if '.' not in domain:
41         return False
42
43     # Domain cannot have consecutive dots
44     if '..' in domain:
45         return False
46
47     # Split domain and check parts
48     domain_parts = domain.split('.')
49
50     # All parts must be non-empty
51     for part in domain_parts:
52         if not part:
53             return False
54
55     # Extension (last part) must be at least 2 characters
56     if len(domain_parts[-1]) < 2:
57         return False
58
59     return True
60
61
62
63
64
65
66
67
68
69
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER ⓘ

OBSERVATIONS

=====

- ✓ Zero-shot: Simple but misses critical validation rules
- ✓ Few-shot: More reliable due to specific examples and requirements
- ✓ Few-shot provides better edge case handling
- ✓ Multiple examples guide Copilot to generate accurate logic

Zero-Shot vs Few-Shot Email Validator (Main comparison)

Zero-Shot Approach

- **Function:** validate_email_zero_shot(email)
- **Logic:** Basic check - just looks for @ and .
- **Accuracy:** ~44% (misses edge cases)
- **Limitation:** Too generic, fails on invalid formats

Few-Shot Approach

- **Function:** validate_email_few_shot(email)
- **Logic:** Comprehensive validation:
 - Exactly one @ symbol
 - Non-empty username and domain
 - No consecutive dots
 - Domain must have at least one dot
 - Extension must be ≥ 2 characters
- **Accuracy:** ~100% (handles all cases)
- **Advantage:** Multiple examples guide accurate code generation

Task 5: Prompt Tuning – Summing Digits of a Number

Task Description #5

- Experiment with 2 different prompt styles to generate a function

that returns the sum of digits of a number.

Style 1: Generic task prompt

Style 2: Task + Input/Output example

Expected Output #5

- Two versions of the `sum_of_digits()` function.
- Example Output: `sum_of_digits(123)` → 6
- Short analysis: which prompt produced cleaner or more optimized code and why?

```
C:\Users\prade > OneDrive > Pradeep > reverse.py > sum_of_digits_v1
47 if __name__ == "__main__":
48     print("-" * 70)
49     print("SUM OF DIGITS - VERSION COMPARISON")
50     print("-" * 70)
51
52     test_numbers = [123, 456, 789, 0, 1, 99, 1000, 12345]
53
54     print("\n--- VERSION 1: String Conversion Approach ---")
55     print("Method: Convert to string, iterate through digits, sum them")
56     print("Pros: Concise, Pythonic, readable")
57     print("Cons: Creates intermediate string object")
58     print(f"\n{'Number':<15} {'Result':<10}")
59     print("-" * 70)
60     for num in test_numbers:
61         result = sum_of_digits_v1(num)
62         print(f"{num:<15} {result:<10}")
63
64     print("\n--- VERSION 2: Mathematical Approach ---")
65     print("Method: Use modulo (%) and division (//) operations")
66     print("Pros: No string conversion, memory efficient")
67     print("Cons: Slightly more complex logic")
68     print(f"\n{'Number':<15} {'Result':<10}")
69     print("-" * 70)
70     for num in test_numbers:
71         result = sum_of_digits_v2(num)
72         print(f"{num:<15} {result:<10}")
73
74     # Verification both versions produce same results
75     print("\n--- VERIFICATION ---")
76     print("Checking if both versions produce identical results:")
77     all_match = all(sum_of_digits_v1(n) == sum_of_digits_v2(n) for n in test_numbers)
78     if all_match:
79         print("✓ Both versions produce identical results!")
80     else:
81         print("✗ Results differ between versions")
82
83     # Detailed example
84     print("\n" + "-" * 70)
85     print("DETAILED EXAMPLE: sum_of_digits(123)")
86     print("-" * 70)
87     num = 123
88     print(f"Version 1 (String Conversion):")
89     print(f"    sum_of_digits_v1(123) = {sum_of_digits_v1(num)}")
90     print(f"    Breakdown: '1' + '2' + '3' = 1 + 2 + 3 = 6")
91
```

```

C:\Users> prade > OneDrive > Pradeep > reverse.py > sum_of_digits_v1
1 # =====
2 # SUM OF DIGITS - VERSION 1 (Using String Conversion)
3 # =====
4 def sum_of_digits_v1(n):
5     """
6     Calculate the sum of digits using string conversion.
7
8     Args:
9         n (int): The number to sum digits from.
10
11     Returns:
12         int: The sum of all digits.
13
14     Example:
15         sum_of_digits_v1(123) -> 6 (1 + 2 + 3)
16         sum_of_digits_v1(456) -> 15 (4 + 5 + 6)
17     """
18     n = abs(n) # Handle negative numbers
19     return sum(int(digit) for digit in str(n))
20
21
22 # =====
23 # SUM OF DIGITS - VERSION 2 (Using Mathematical Operations)
24 # =====
25 def sum_of_digits_v2(n):
26     """
27     Calculate the sum of digits using modulo and division operations.
28
29     Args:
30         n (int): The number to sum digits from.
31
32     Returns:
33         int: The sum of all digits.
34
35     Example:
36         sum_of_digits_v2(123) -> 6 (1 + 2 + 3)
37         sum_of_digits_v2(456) -> 15 (4 + 5 + 6)
38     """
39     n = abs(n) # Handle negative numbers
40     total = 0
41     while n > 0:
42         total += n % 10 # Add the last digit
43         n //= 10 # Remove the last digit
44     return total
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96 print("\n" + "=" * 70)
97 print("COMPARISON SUMMARY")
98 print("=" * 70)
99 print("✓ Both versions handle the same problem differently")
100 print("✓ Version 1 is more readable and concise")
101 print("✓ Version 2 is more algorithmic and memory-efficient")
102 print("✓ Choose based on performance needs and code readability preference")
103

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
Version 1 (String Conversion):
sum_of_digits_v1(123) = 6
Breakdown: '1' + '2' + '3' = 1 + 2 + 3 = 6

Version 2 (Mathematical):
sum_of_digits_v2(123) = 6
Breakdown: (123 % 10) + (12 % 10) + (1 % 10) = 3 + 2 + 1 = 6

```

1. **Trade-offs:** Understanding pros/cons helps choose the right implementation